MAC5921 - Deep Learning — EP1

André Nogueira Ribeiro

October 24, 2026

1 Declaração sobre uso de IA

O código implementado para este trabalho foi gerado com ajuda da IA, utilizando uma IDE com modelos integrados. Abaixo se encontra com mais detalhes como foi utilizada a ajuda desta ferramenta:

- Códigos associados às implementções de funções e orquestrações para rodar os experimentos foram feitos com muito cuidado, o que inclui revisões detalhadas das sugestões geradas por IA, discussões com os modelos sobre diferentes possibilidades e razões para certas sugestões.
- Códigos de teste foram revisados de maneira um pouco mais superficial, conferindo se o que estava sendo testado fazia sentido e em alguns casos indo mais a fundo no como isso era feito.
- Códigos para geração de figuras e tabelas com os resultados foram feitos com pouca intervenção da minha parte.
- Este relatório foi feito sem nenhuma ajuda de modelos, exceto para pequenas formatações e sintáxe de LaTeX.

2 Código

O código, incluindo este relatório feito em LaTeX, pode ser encontrado em:

- https://github.com/deconr/MAC5921-DeepLearning.

3 Experimentos

Essa seção descreve os experimentos realizados (incluindo o planejamento) e apresenta os resultados obtidos. Analisarei o valor da função de perda e da acurácia para os conjuntos de treinamento e validação ao longo do treinamento.

Nesse contexto, é importante mencionar que, para o conjunto de treinamento, o valor final é uma média dos valores que são calculado para cada batch com os pesos atuais da rede (atualizados após cada batch), ou seja, cada valor de loss/acc é calculado para rede com pesos diferentes dentro de uma mesma época. Já para o conjunto de validação, o valor é calculado com os pesos congelados ao final da época. Isso explica o porquê os gráficos referentes ao treinmento iniciam com valores piores em relação a métrica avaliada. Por exemplo, no gráfico Loss / Epoch, o valor inicial das curvas de treinamento são bem maiores que o valor inicial das curvas de validação.

3.1 Aspectos gerais

A ideia desse experimento é comparar NNs com CNNs, considerando aspectos gerais, sem modificar o dataset utilizado. Para isso, construí modelos com números similares de parâmetros e analisei o comportamento durante o treinamento e validação (results/curves.png). O resultado nos dados de teste também será mencionado.

Análise	**Treinamento**	**Validação**	**Considerações**
Variação da loss ao longo das épocas (curves.png). CNN com 52,138 parametros. NN com 52,480 parametros Mais detalhes da arquitetura no arquivo models.py.	Cai muito nas duas primeiras épocas, o que é esperado dado que os pesos da rede começam com valores aleatórios. Percebe-se que já na primeira época a CNN associa-se a uma loss menor em relação a NN, o que continua sendo verdade por todo o treinamento. Também é possível perceber que a CNN converge mais rápido, o que reflete uma inclinação menor mais rapidamente quando comparada a curva da NN.	Cai menos do que as curvas de treinamento nas primeiras épocas (devido ao que foi explicado acima), porém a comparação também mostra que a CNN se associa a uma loss menor em relação a NN por todo o treinamento. A única diferença é que na validação as curvas convergem mais rápido e em tempos parecidos porém para um valor mais alto. A curva da CNN é levemente mais suave.	As análises do resultado confirmam o que já era esperado: a CNN teve um desempenho melhor, o que normalmente é atribuído ao fato de que a rede leva em considerção a localidade (convoluções que analisam pixels vizinhos), o que é uma caracteristica dos dados analisados (imagens). Concordando com o que foi mencionado, a acurácia final também foi melhor para CNN, 0.9879 contra 0.9728 (summary.json)

3.2 Ruído gaussiano no conjunto de teste

O objetivo desse experimento é analisar a robustez dos modelos CNN e NN. A ideia é medir o quanto a acurácia cai quando adicionamos ruído nas imagens do conjunto de teste.